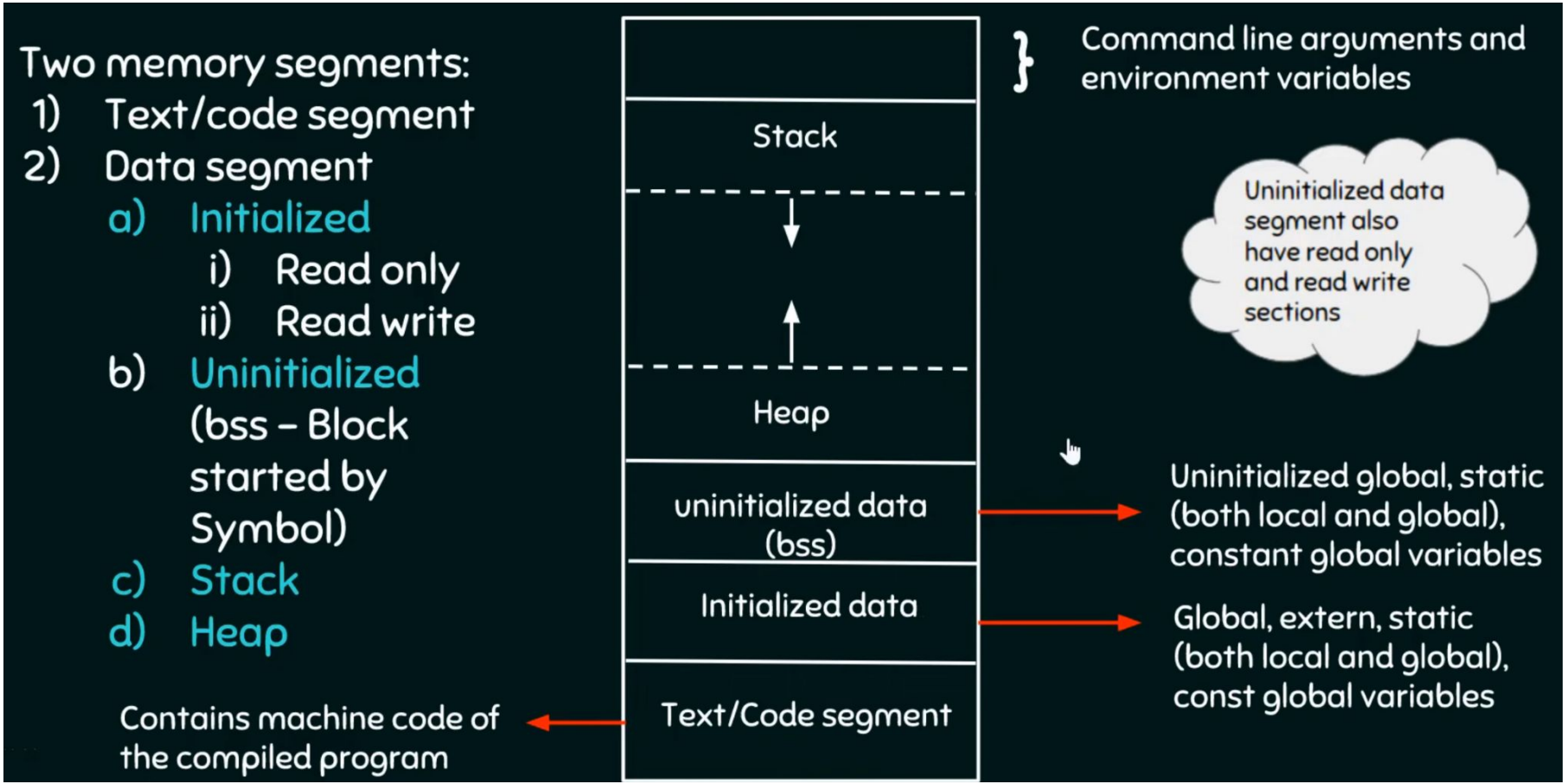
Concepts of Programming Language

Variable Scopes

Source:
https://youtu.be/gegaS_gX3TY?si=DKczIIrykisV0DWW
https://youtu.be/L53nqHCSSFY?si=fMoLy2am8ZCiot5G

Memory layout of C Program



Memory layout of C Program

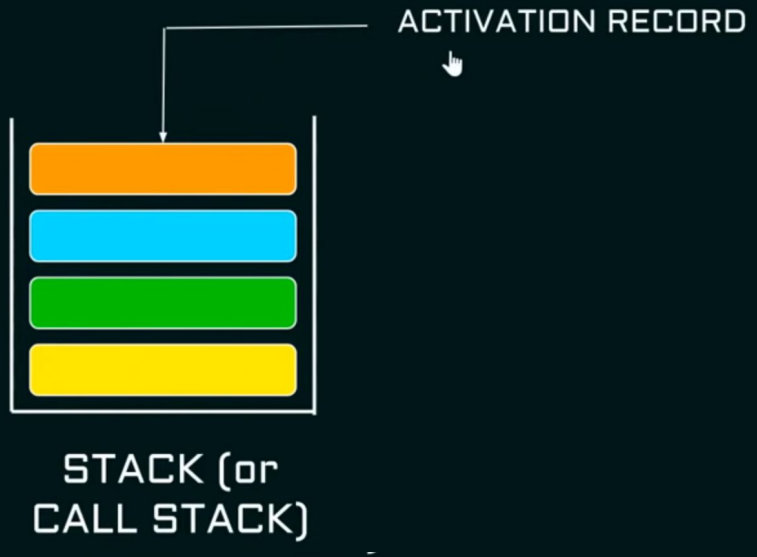
```c
#include <stdio.h>

static int i;
static int i = 27;
static int i;
int main() {
    static int i;
    printf("%d", i);
    return 0;
}
```

a) 27
b) 0
c) No output
d) None of the above

Stack memory segment

- Stack is a container (or memory segment) which holds some data.
- Data is retrieved in Last In First Out (LIFO) fashion.
- Two operations: push and pop.

```
main()
{
    fun1();
}

fun1() { fun2(); }
fun2() { fun3(); }
fun3() { return; }
```



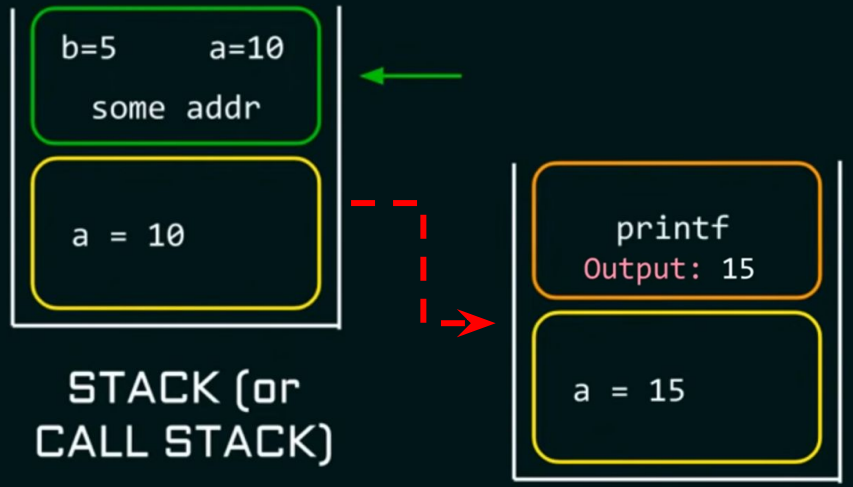
Stack memory segment

Activation Record (▭) – is a portion of a stack which is generally composed of:

1. Locals of the callee
2. Return address to the caller
3. Parameters of the callee

Example:

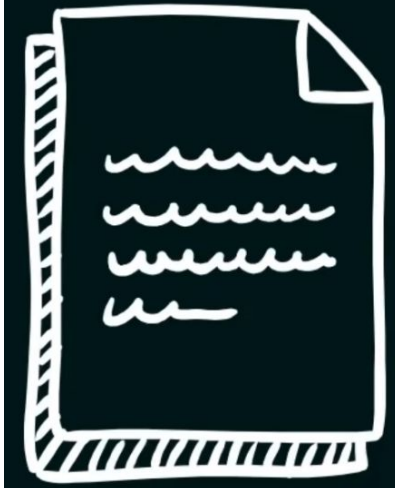
```
int main()
{
      int a = 10;
      a = fun1(a);
      printf("%d", a);
}

int fun1(int a)
{
      int b = 5;
      b = b+a;
      return b;
}
```



Variable Scoping

- The *scope* of a variable is the range of statements over which it is visible
- The *local variables* of a program unit are those that are declared in that unit
- The *nonlocal variables* of a program unit are those that are visible in the unit but not declared there
- *Global variables* are a special category of nonlocal variables
- The scope rules of a language determine how references to names are associated with variables

Why Scoping



Scoping is necessary if you want to reuse variable names

Example:

```
int fun1()
{
     int a = 10;
}

int fun2()
{
     int a = 40;
}
```

Static Scoping

In Static scoping (or lexical scoping), definition of a variable is resolved by searching its containing block or function. If that fails, then searching the outer containing block and so on.

```
int a = 10, b = 20;
int fun()
{
    int a = 5;
    {
        int c;
        c = b/a;
        printf("%d", c);
    }
```

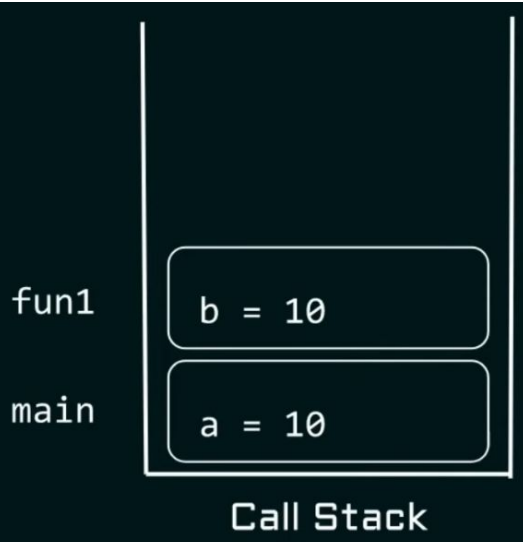
Output: 4

Static Scoping

```
int fun1(int);
int fun2(int);
int a = 5;
int main()
{
    int a = 10;
    a = fun1(a);
    printf("%d", a);
}
```

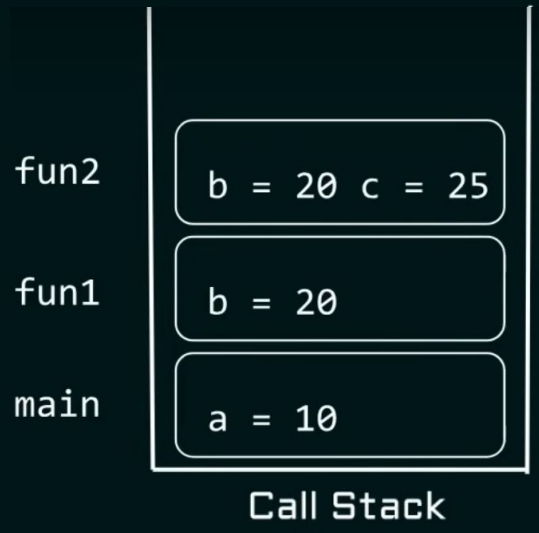
```
int fun1(int b)
{
    b = b+10;
    b = fun2(b);
    return b;
}
```

```
int fun2(int b)
{
    int c;
    c = a + b;
    return c;
}
```



Static Scoping

```
int fun1(int);
int fun2(int);
int a = 5;
int main()
{
    int a = 10;
    a = fun1(a);
    printf("%d", a);
}

int fun1(int b)
{
    b = b+10;
    b = fun2(b);
    return b;
}

int fun2(int b)
{
    int c;
    c = a + b;
    return c;
}
```

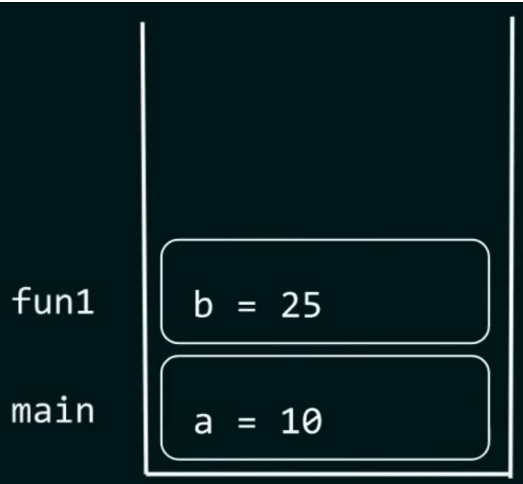


Static Scoping

```
int fun1(int);
int fun2(int);
int a = 5;
int main()
{
    int a = 10;
    a = fun1(a);
    printf("%d", a);
}
```

```
int fun1(int b)
{
    b = b+10;
    b = fun2(b);
    return b;
}
```

```
int fun2(int b)
{
    int c;
    c = a + b;
    return c;
}
```

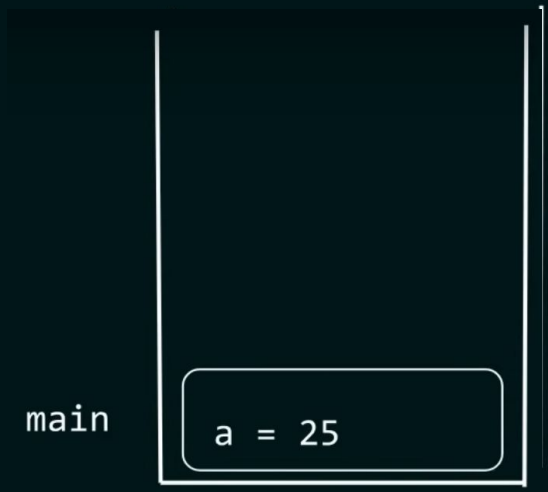


Static Scoping

```
int fun1(int);
int fun2(int);
int a = 5;
int main()
{
    int a = 10;
    a = fun1(a);
    printf("%d", a);
}

int fun1(int b)
{
    b = b+10;
    b = fun2(b);
    return b;
}
```

```
int fun2(int b)
{
    int c;
    c = a + b;
    return c;
}
```

Output: 25



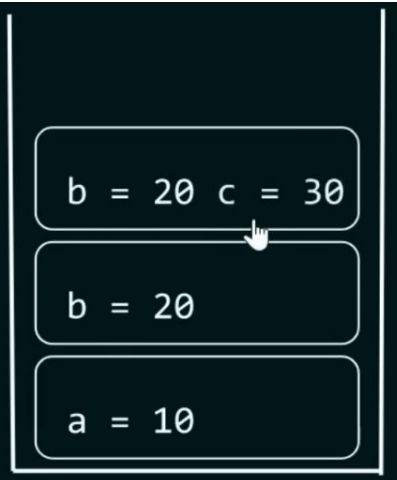
Evaluation of Static Scoping

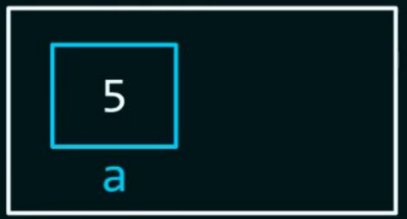
- Works well in many situations
- Problems:
  - In most cases, too much access is possible
  - As a program evolves, the initial structure is destroyed and local variables often become global; subprograms also gravitate toward become global, rather than nested

Dynamic Scoping

```
int fun1(int);
int fun2(int);
int a = 5;
int main()
{
    int a = 10;
    a = fun1(a);
    printf("%d", a);
}

int fun1(int b)
{
    b = b+10;
    b = fun2(b);
    return b;
}
```

```
int fun2(int b)
{
    int c;
    c = a + b;
    return c;
}
```





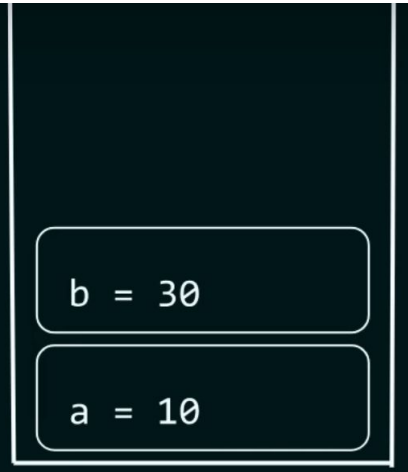
Dynamically scoped programming languages include bash, LaTeX, and the original version of Lisp

In dynamic scoping, definition of a variable is resolved by searching its containing block and if not found, then searching its calling function and if still not found then the function which called that calling function will be searched and so on.

Dynamic Scoping

```
int fun1(int);
int fun2(int);
int a = 5;
int main()
{
    int a = 10;
    a = fun1(a);
    printf("%d", a);
}

int fun1(int b)
{
    b = b+10;
    b = fun2(b);
    return b;
}
```

```
int fun2(int b)
{
    int c;
    c = a + b;
    return c;
}
```



Dynamic Scoping

```
int fun1(int);
int fun2(int);
int a = 5;
int main()
{
    int a = 10;
    a = fun1(a);
    printf("%d", a);
}

int fun1(int b)
{
    b = b+10;
    b = fun2(b);
    return b;
}
```

```
int fun2(int b)
{
    int c;
    c = a + b;
    return c;
}
```

Output: 30

Evaluation of Dynamic Scoping

- Advantage: convenience
- *Disadvantages:*
  1. While a subprogram is executing, its variables are visible to all subprograms it calls
  2. Impossible to statically type check
  3. Poor readability- it is not possible to statically determine the type of a variable

Homework Problem

What will be the output of the following program snippet:

```
int a,b;

void print() {
    printf("%d %d", a, b);
}

int fun1() {
    int a, c;
    a = 0; b = 1; c = 2;
    return c;
}

void fun2() {
    int b;
    a = 3; b = 4;
    print();
}

int main() {
    a = fun1();
    fun2();
}
```

With static scoping

a) 2 4
b) 3 1
c) 2 5
d) 3 4

With dynamic scoping

a) 2 4
b) 3 1
c) 2 5
d) 3 4

Static memory allocation

Memory allocated during compile time is called static memory.

The memory allocated is fixed and cannot be increased or decreased during run time.

EXAMPLE:

```
int main()
{
     int arr[5] = {1, 2, 3, 4, 5};
}
```

Memory is allocated at compile time and is fixed.

Static memory allocation – Drawbacks

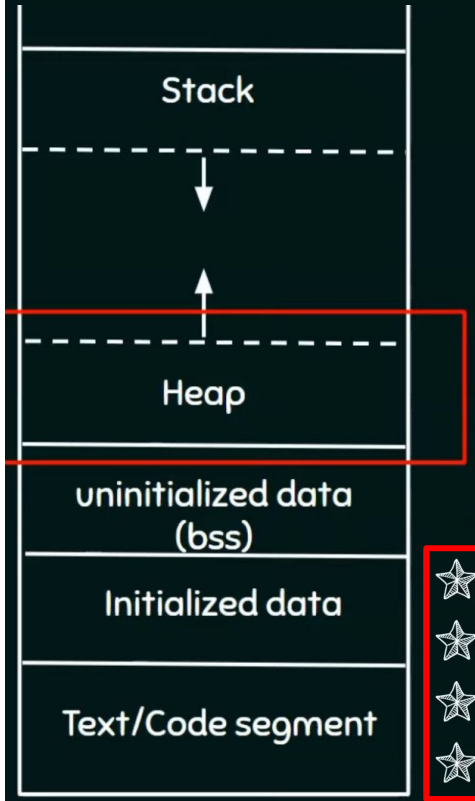
If you are allocating memory for an array during compile time then you have to fix the size at the time of declaration. Size is fixed and user cannot increase or decrease the size of the array at run time.

If the values stored by the user in the array at run time is less than the size specified then there will be wastage of memory.

If the values stored by the user in the array at run time is more than the size specified then the program may crash or misbehave.

Dynamic memory allocation

The process of allocating memory at the time of execution is called **dynamic memory allocation.**



Heap is the segment of memory where dynamic memory allocation takes place

Unlike stack where memory is allocated or deallocated in a defined order, heap is an area of memory where memory is allocated or deallocated without any order or randomly.

- `malloc()`
- `calloc()`
- `realloc()`
- `free()`

There are certain built-in functions that can help in allocating or deallocating some memory space at run time.